Constructing a tamper evident synthesis log for benchtop DNA synthesisers

Rio Thomas
University of Cambridge
rt640@cam.ac.uk

Mackenzie Noon
Yale University
mackenzie.noon@yale.edu

***Abstract*—Today benchtop DNA synthesisers can produce strands of 200 nucleotides. Improving enzymatic techniques and hardware automation will reduce the cost of using benchtop devices for DNA synthesis, promoting their widespread use. Secure logging is required to hold the device owners accountable for the synthesized DNA, including sequences of concern. Current benchtops log synthesis runs, but an adversary who wishes to produce a sequence of concern may try to tamper with logs so as to evade detection. Here, we specify the design of a synthesis log based on Merkle trees to ensure integrity while reducing the exposure of proprietary information during an audit. We outline the security guarantees provided by hardware level protections and by external records kept off the device through a partial cloud connection.**

1. Introduction

Computational and deep learning tools speed up the generation and prioritization of plausible protein candidates, creating demand for synthesized DNA and empirical testing [1]. Along with the increasing demand for synthetic DNA, its accessibility also increases, with many companies providing synthetic DNA at a cost of approximately $0.07–0.15 per base pair (bp). While the cost of ordering DNA from specialized vendors is reasonably low, benchtop DNA synthesizers may offer increased privacy and faster on-demand access [2]. Benchtop devices that can synthesize short single stranded DNA oligonucleotides (nt) have existed since the 1980s [3]. Benchtop DNA-synthesis and assembly platforms are likely to become more capable and broadly accessible as enzymatic chemistry, automation, and integrated biofoundry workflows mature [4]. Improved hardware automation with liquid handling robots, microfluidic chips and electrowetting will make DNA synthesis even faster and more precise. Metaculus forecasts that benchtops that can produce 5 kbp fragments will exist by 2030, and cost about 190,000 USD; and only about a hundred benchtops have been sold globally that can synthesize fragments up to 1kbp [2].

David Baker and George Church [5] argue that the insufficiency of screening techniques necessitates the logging of synthesized sequences whilst maintaining trade secrets. This enables DNA traceback regimes from global threats, especially considering that due to their complexity, dangerous agents may take many attempts to create. A 'selective revelation' policy can allow compromising queries in exceptional circumstances. The awareness of an audit trail also deters bad actors. Similar ideas are echoed in the open letter signed by several leaders of major AI companies [6] calling for mandatory synthesis screening and record keeping in the United States. So far, the focus has remained on employing these strategies to centralized synthesizers, but this paper aims to outline the same private logging function with a 'selective revelation' policy for the incoming benchtop paradigm.

Regulation is slowly emerging. Both the October 2023 Human and Health Services Screening Framework Guidance [7] and UK screening guidance on synthetic nucleic acids [8] recommend implementing a data logging function on their devices to maintain a record of nucleic acids synthesized on equipment. Retained records can also identify split order attacks. Current devices do track synthesis requests but not in a way that is resistant to tampering.

This paper outlines the construction of a data logging function that is resistant to tampering. We propose an auditing authority which performs routine audits at regular intervals, without exposure of proprietary intellectual information, alongside an external trusted server. We also aim to minimise any internet connection requirements, mitigating the possibility of data leakage whilst avoiding the persistent offline access which might allow an advanced actor to create sequences of concern without their being logged. We discuss the current landscape of internet connectivity of benchtops, the ideal properties of these devices, the threat models we are worried about, our system design, an auditing scheme and a final discussion of the security guarantees left unaddressed.

2. Background

The Nuclear Threat Initiative's benchtop report [9] lays out two options for screening DNA sequences on benchtop devices: on-device or in the cloud. The U.S. Department of Health and Human Services has published guidelines [7] that recommend screening against sequence of concern (SOC) databases regularly updated as new SOCs are identified, requiring an external connection. Even if screening occurs locally, the authorisation of users, whether one time or expiring, will happen through a connection to a centralised authority [9]. Where authorisation is denied or the connection is broken, synthesis is refused [9]. A capable

operator can bypass such a denial and continue to use the device, or obtain authorisation under false pretences. Regulatory agencies may additionally permit wholly offline access to entities judged sufficiently trustworthy [7]. Each of these routes leaves an operator in continuous possession of an unmonitored, working synthesiser. We assume current benchtops do not enforce append-only at the hardware level, so logs can be modified by a competent actor [9]. If the benchtop has an uninterrupted connection to the cloud, then a real time backup of the logs can be stored on a trusted server, and tampering can be trivially detected. A malicious operator will attempt to break any external connection so that this does not occur — a benchtop has its weakest security guarantees when completely offline. Connected systems can notify relevant authorities when the online connection of a benchtop has been broken [10]. When wholly offline, it is difficult to prevent an operator from tampering with any entry, deleting, editing or adding forged ones.

However, despite the many advantages of an online system, there are several reasons to keep benchtops offline. A connection to an external screening service renders the system vulnerable to cyberattacks [11]. If sequences are proprietary intellectual property or protected patient sequences, synthesis providers may not want them to leave their device [12]. There are also scenarios which require a benchtop to be fully offline, e.g. low-resource settings and fully airgapped labs, but we expect such circumstances to be rare, and well covered by existing approaches. For example, SecureDNA deploys a solution [13] for airgapped benchtops where permission to synthesise a given DNA order is provisioned by a centralised screening server. The server returns a digital certificate tied to a hardware token that has to be brought physically into the air-gapped facility. Tampering with a synthesis log is not a concern here, as we would be highly confident that every sequence being synthesised has passed screening. Instead, if we rely solely on screening software that is local to the device, then the synthesis log is easier to inappropriately modify. Regardless of the exact screening architecture, it is desirable that some screening result for a synthesis request is stored on a tamper evident log alongside the sequence.

We assume the system has a number of software protections like OS access control (Unix DAC) [14] and append-only attributes. These defenses are defeated by an operator with full physical access to the machine. For devices with online connections, remote logging [10] is an option, where synthesis logs are stored off the device. Integrity monitoring tools like AIDE and tripwire [15] detect changes to files by computing their hashes. Benchtops will likely have systems with a secure boot [16], preventing unauthorised kernels from loading. Trusted Platform Modules (TPMs) could be used to ensure the integrity of the boot process by signing values with a private key [17].

3. Preliminaries

Let $\mathcal{O}$ be an adversarial operator in possession of a benchtop DNA synthesiser, $\mathcal{B}$, released by manufacturer, $\mathcal{M}$. When $\mathcal{O}$ synthesises a sequence, an entry is appended to the log. An external anchor of root hashes, $\mathcal{R}$, is kept secure by a trusted party, $\mathcal{T}$, that can either be $\mathcal{M}$ or $\mathcal{A}$. An auditor, $\mathcal{A}$ performs intermittent audits, checking if the synthesis log has been tampered with and whether any SOCs have been synthesised.

3.1. Assumptions

A1. *Hash function security.* It is infeasible to find the input to a hash function, SHA-256 [18], based on its output, or to find two different inputs with the same output.

A2. *Signature unforgeability.* A valid signature over a message is unique to the private signing key [19] held in the secure element and to the message.

A3. *Decrypting key privacy.* This is held in a trusted location and only accessible to $\mathcal{T}$.

TABLE 1. ATTACKER ACCESS TYPES [13]

Access type	Description
Remote, limited	A remote attacker may have limited time, because the exploit relies on time-sensitive vulnerabilities that have since been patched, or because administrators may observe unusual activity.
Remote, persistent	Uninterrupted, undetected access to a synthesiser's network, e.g. through a compromised connected device or a remote code execution vulnerability.
Limited physical access	Access only for certain hours (e.g. a student in an academic lab with access outside working hours).
Persistent physical access	The attacker has purchased the machine and has unlimited, continuous access including reagents — via persistent impersonation or authorised misuse.

TABLE 2. ATTACKER SKILL LEVELS [13]

Skill level	Description
Novice	Minimal technical skill and biology knowledge; may attempt manual tampering with the device.
Competent	Able to use automated scripts and public CVEs. Budget under $1,000.
Moderate	Able to identify and exploit common software vulnerabilities and basic hardware attacks. Budget up to $10,000.
Advanced	Custom malware and sophisticated side-channel monitoring. Budget up to $100,000.

3.2. Threat model

We are most concerned about the advanced, malicious $\mathcal{O}$ with persistent physical access to $\mathcal{B}$. We are also interested in the moderate actor with limited physical access to $\mathcal{B}$: an adversary operating at a location they do not own, with

access only for part of the time. The threat is that they sever the partial connection and synthesise something before $\mathcal{B}$ shuts down, or avoid the shutdown entirely. A merely competent actor can disrupt the partial connection but cannot avoid the shutdown of $\mathcal{B}$.

3.3. Enforcement properties

Secure logging would ideally rest on the following enforcements.

E1. *Signing key not extractable.* The signing key held in the secure element is not accessible to $\mathcal{O}$ [20].

E2. *Oracle access limited.* The secure element signs only values meeting defined criteria, so O cannot obtain a signature over an arbitrary message.

E3. *Stable connectivity.* $\mathcal{B}$ has a partial connection to a trusted location which cannot be broken by $\mathcal{O}$.

E4. *append-only enforcement.* Hardware constrains $\mathcal{O}$ to only appending entries to log [13].

E5. *Synthesiser hardware tamper-proof.* The connection between the synthesiser valves and the microprocessor is secure so that the log accurately records the sequence being synthesised [13].

3.4. Attacks

An advanced actor with prolonged physical access to the benchtop can break all of our enforced properties.

Instead of tampering with logged entries, false novel entries can be appended to the log by violating E5, the tamper evidentness of synthesis hardware through multiple methods such as altering packets in transmission, swapping nucleotide wells, directly communicating with JTAG and UART pins [13]. These attacks can be mitigated through anti-tamper measures on devices, such as tamper evident seals with warranty void notices, secure hardware interfaces, and measures to prevent nucleotide well swapping [13]. The exact implementation of such systems lies outside the scope of the present work.

$\mathcal{B}$ can establish a connection over a network $\mathcal{O}$ controls, e.g. a lab router. This connection can be broken easily and masked as a network failure by a moderately advanced $\mathcal{O}$. With E3, stable connectivity, violated and hardware attacks that can break E2, limited oracle access, and E4, append-only enforcement, $\mathcal{O}$ can edit entries appended to the log after the connection is disrupted, evading any remote logging approaches to store synthesised sequences online. If the external connection is made through a channel the operator doesn't control, e.g. an embedded SIM, then a greater effort is required to break E3 and it isn't possible to deny the attempt.

In the presence of a CPU, if E4 is enforced by file append-only OS attributes, anyone with root access can change file permissions [14], breaking E4. The manufacturer should withhold the root from $\mathcal{O}$. However $\mathcal{O}$ can attack known software vulnerabilities or extract the root credentials by other means. During the boot process, there are stages where append only restrictions for files are not enforced [16]. An adversary could modify the synthesis log by interrupting the boot process, in which case a secure boot is needed. Ultimately, $\mathcal{O}$ can reflash the software so that they control the root and can modify append only permissions. Trusted Platform Modules(TPMs) can also protect against the reflashing of firmware [17], slightly raising costs. But we are unable to prevent the reflashing of firmware given enough effort by an advanced actor, e.g. by desoldering the chip, programming it externally and resoldering it. Similarly, E2, limited oracle access such that the secure element will only sign entries meeting specific criteria, is a solution requiring logic on the firmware level which is vulnerable to reflashing.

The signing key itself could be extracted, breaking E1, by sophisticated means like side channel analysis or microprobing internal buses [21]. This would allow third parties to present their own versions of logs with valid signatures belonging to $\mathcal{O}$.

The tampering of a log or its truncation, are both software attacks $\mathcal{O}$ can perform if they have the root of trust of the system and there are no cryptographic or hardware defences.

The integrity of the external anchor can be broken by an advanced actor hacking into the trusted location $\mathcal{T}$, and presenting an altered version of the external anchor as the valid version.

4. Design

During an audit to check for the tampering of a synthesis log, there may be a significant breach of intellectual property. Our log structure must minimise the information that an auditor needs to confirm that there has been no tampering.

We expect a common use-case to be a forensic auditor who has a partial sequence from an incident and is looking for a similar sequence synthesized on a benchtop. This requires that the encrypted raw sequences be stored on the benchtop device rather than their hashed commitments like in a SecureDNA privacy preserving screening system [12]. Functional screening performed locally on benchtop devices requires full sequences regardless [5].

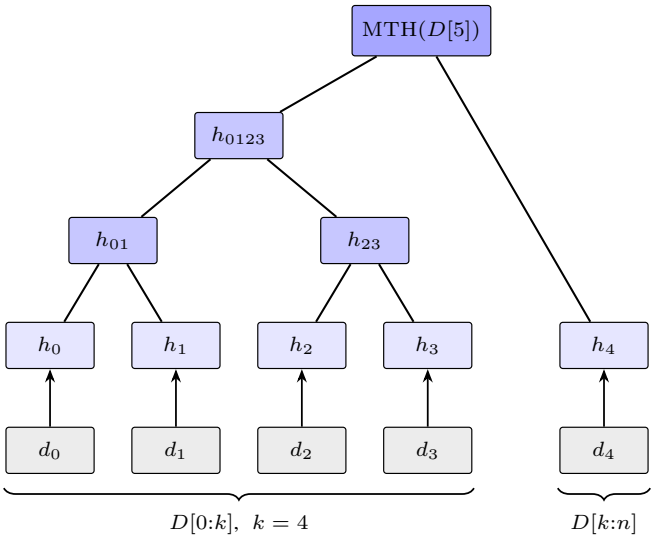
4.1. Data structure

A Merkle tree [22] takes its leaves as the hash of each entry in the synthesis log as shown in Figure 1. Binary pairs of leaf hashes are combined and hashed to form the next layer in the tree. This repeats until a single tree hash is obtained. If any entry in the log is edited or deleted, the tree hash changes and we detect tampering. To check if the log has been tampered with, we only need to perform *inclusion* and *consistency* proofs [23]. An inclusion proof checks if a certain entry claiming to belong to the log actually does and a consistency proof confirms that a later version of the tree

is a valid extension of an earlier one. These proofs require audit paths - the shortest list of nodes required to compute the tree hash from a given leaf hash. In Figure 1, the audit path for h_2 is [h_3, h_{01}, h_4]. Significantly, an auditor given this audit path can infer nothing about the raw sequences stored in the log, motivating the use of a Merkle tree over a hash chain. In the latter, tampering is confirmed by checking the integrity of the chain and accessing the entries in each block, revealing intellectual property.

4.1.1. Notation.

H - Hash function, SHA-256.
$\|$ - Concatenation of byte strings.
`0x00`−, `0x01` Single-byte domain-separation prefixes, applied to leaf and interior inputs respectively.
$D[n]$ - The log, an ordered list of n entries $d(0), \ldots, d(n-1)$.
$D[i{:}j]$ - The sublist $d(i), \ldots, d(j-1)$, containing $j-i$ entries.
n - The number of entries in the log, referred to as the *tree size*.
k - For $n > 1$, the largest power of two strictly less than n, so that $0 < k < n$.
$\mathrm{MTH}(D[n])$ - The Merkle Tree Hash of $D[n]$, a single value of H's output length committing to every entry in the list.



leaf: $h_i = H(\texttt{0x00} \,\|\, d_i)$ interior: $H(\texttt{0x01} \,\|\, \text{left} \,\|\, \text{right})$

Figure 1. Merkle Tree Hash over $n = 5$ entries, following RFC 9162 §2.1.1. The list is split at k, the largest power of two strictly less than n; the left subtree is therefore always perfect and the right subtree recurses. The tree is not padded to a power of two, so h_4 is promoted directly to the root and the shape is left-heavy by construction. Distinct prefixes `0x00` and `0x01` separate the leaf and interior domains, preventing a leaf hash from being substituted for an interior node.

4.1.2. Log entry.

Each entry has the following core fields:

```
{
  "index": 7,
  "timestamp": "2026-07-13T14:22:01Z",
  "seq_ciphertext": "8Kx2...q9Fw",
  "metadata": metadata
}
```

In the case that multiple users are operating $\mathcal{B}$, we expect the user ID to be included in the metadata of an entry so that entries across all users are stored in one tree rather than individual ones for ease of auditing. If users want a history of their own orders, a separate list can be maintained for this purpose that is not tamper evident.

To ensure that entries with the same information don't map to different hashes due to deviations in formatting, we define a canonicalisation protocol.

S1. Sort the fields in predetermined order
S2. Ensure entries remain in ASCII
S3. Write sequence in upper case
S4. Whitespace/newline stripping
S5. Sequence is made of IUPAC nucleotide codes and ambiguity codes are kept as they are.

4.1.3. Tree construction.

Merkle trees are defined rigorously in RFC 9162 §2.1.1 [24], with SHA-256 as the hash function. An exact formulation is provided below:

$$\begin{aligned} \mathrm{MTH}(\{\}) &= \mathrm{HASH}() \\ \mathrm{MTH}(\{d[0]\}) &= \mathrm{HASH}(\texttt{0x00} \,\|\, d[0]) \\ \mathrm{MTH}(D[n]) &= \mathrm{HASH}\big(\texttt{0x01} \,\|\, \mathrm{MTH}(D[0{:}k]) \\ &\qquad \|\ \mathrm{MTH}(D[k{:}n])\big) \end{aligned} \tag{1}$$

where $n > 1$ and k is the largest power of two strictly less than n, so that $0 < k < n$.

4.2. External Anchor

The signed tree hashes are stored in an external anchor, $\mathcal{R}$, with a trusted authority $\mathcal{T}$ containing the following fields after passing through a similar canonicalisation process as to Section 4.1.2, without steps S3–S5.

```
{
  "device_id": 'u3nf84fj9',
  "tree_size": 5,
  "root_hash": "6139df0e5f71...4aedd7",
  "timestamp": "2026-07-30T09:58:20Z",
  "signature": "8d2998aa983...c30c49d"
}
```

As stated in Section 3, $\mathcal{T}$ can either be the manufacturer $\mathcal{M}$ or a third party auditing authority. $\mathcal{M}$ will already have a connection to the device for firmware updates and changing the table of SOCs, however our scheme does not require that they be trusted. Similar to certificate transparency logs [23], a log split across multiple locations can hold the anchor. This infrastructure doesn't exist for

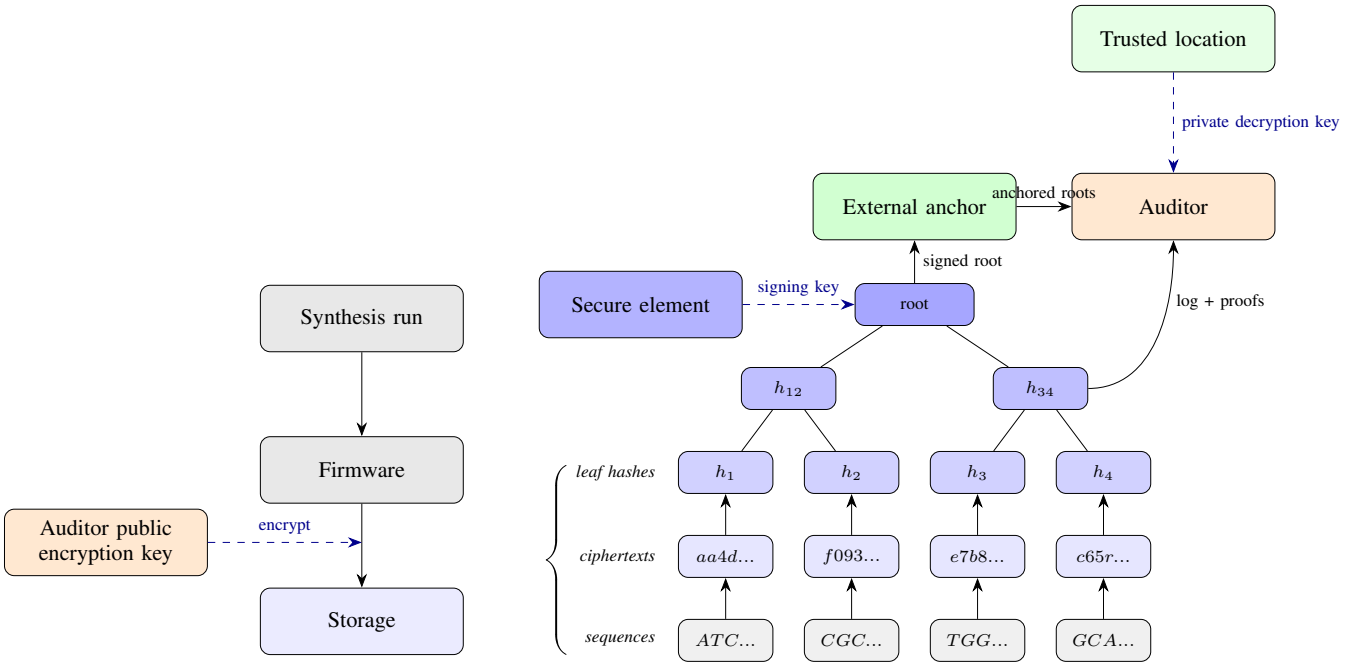


Figure 2. Log construction and verification. Each sequence is encrypted under the auditor's public key; hashes of the resulting log entries form the leaves of a Merkle tree whose root is signed by the secure element and published to an external anchor. An auditor verifies integrity against anchored roots and recovers sequence content using the private decryption key held at a trusted location. Routine integrity verification re-hashes stored ciphertext and never requires decryption.

DNA synthesis screening yet but may in the future when cross-provider registers are created to detect split-order attacks [25]. We require that $\mathcal{O}$ receive a receipt from $\mathcal{T}$ with a signature over the tree hashes they append to $\mathcal{R}$. Ideally, the external connection is made through a channel the operator doesn't control, e.g. an embedded SIM.

4.2.1. Receipt protocol. Let $\mathcal{R}$, be updated once every T with a list of root hashes corresponding to each of X appends in time, T. For each added entry, $\mathcal{T}$ returns a receipt of the form:

```
{
  "device_id": 'u3nf84fj9',
  "tree_size": 5,
  "root_hash": "6139df0e5f71...4aedd7",
  "receipt_timestamp": "2026-07-30T09:58:24Z",
  "anchor_signature": "b71c04ee2a1...f882e13"
}
```

Once again, canonicalisation is applied. When $\mathcal{R}$ is updated with the X root hashes, $\{R_n, R_{n+1}, \ldots, R_{n+x}\}$, X receipts are sent back to $\mathcal{O}$. A verification is done to ensure receipts are signed over the actual root hashes, $\{R_n, R_{n+1}, \ldots, R_{n+x}\}$. If this isn't the case, we assume an error has occurred because $\mathcal{T}$ is trusted and signing is redone.

4.2.2. Anchoring cadence
. If $\mathcal{R}$ is updated every T, then the entries synthesised in the last time period T are unanchored and vulnerable to modification. If λ sequences are made per unit time, then the expected maximum of entries at exposure to modification is λT. This must be considered when deciding on connection rates for $\mathcal{B}$. If a connection cannot be established, this warrants escalation to investigate $\mathcal{O}$.

4.3. Hardware enforcements

Notably, the signing key requires certain properties of the system which can only be enforced with hardware. E1 (preventing extraction) is easily accomplished with a secure element [20]. E2 (preventing oracle abuse) is more challenging. E2 is important because, without any limitations on signing, an adversary can simply sign any Merkle root, allowing them to authenticate logs which have been forged to remove entries. Although this can be mitigated with online checkpoints, the aforementioned intellectual property concerns make an offline, hardware-backed E2 desirable.

For this reason, we include a simple hardware-backed schema to address both E1 and E2. Although it cannot totally prevent oracle abuse, our schema can create an immutable record of oracle use, preventing adversaries from silently editing past logs. This has the additional benefit of providing rollback and forking detection at the hardware level. We propose the use of a Trusted Platform Module non-volatile memory extend index (TPM NV extend index) [17]. Such indices cannot be set to arbitrary data, but instead can only be extended : the `TPM2_NV_Extend` operation performs `new_value=SHA256(old_value || new_data)`, saving only `new_value`. Merkle root hashes

can be passed to this function, and so an immutable commitment of all roots is saved on-device.

During provisioning, a TPM hardware attestation key and an initial value, V_0, are set and registered with the auditor $\mathcal{A}$, and a TPM NV index is created and made permanent by using TPM policy settings to make deletion impossible without an auditor key. During operation, each merkle root is checked in to the TPM by executing `TPM2_NV_Extend`, computing $V_i = SHA256(V_{i-1}||L)$, where the logging information L contains the merkle root hash and the tree size.

As described in the Attacks section, append-only properties enforced by firmware can be circumvented by reflashing. Regardless, firmware-enforced append-only synthesis logs will make tampering more difficult.

Voltage/Temperature/light sensors can detect tampering and cause a device lockout, which requires servicing from the manufacturer to unlock. This shutdown is enforced by firmware and so can be evaded by someone who has compromised the hardware and flashes the device. Physical tampering can also be reported to the manufacturer through an online connection, unless the connection is cut before the incident. The benchtop should be built such that any attempt to flash firmware requires access to the device's hardware, e.g. opening the device or swapping a chip. If anti-tampering measures like tamper seals, sensors and tamper mesh are added [13], any in-person investigation will discover evidence of tampering.

4.4. Signing and Encryption

We use an Ed25519 asymmetric signing scheme [19] to sign every root hash using a private key generated in the secure element and a public key held by $\mathcal{T}$. A hybrid public key encryption scheme is used to store ciphertext sequences on $\mathcal{B}$, with the public encrypting key held on device and private decrypting key held by $\mathcal{T}$.

5. Auditing

5.1. Routine Audits

Routine audits check for tampering without revealing specific nucleotide sequences. If tampering is detected, a forensic audit can be performed to check for SOCs. 'AuditInterval' is the time period after which a routine audit by $\mathcal{A}$ occurs. It determines how much time a malicious actor has to act undetected after synthesizing a dangerous sequence. The upper bound for the auditing frequency is set by the anchoring frequency. Auditing can be performed on the same partial connection as anchoring. A routine audit consists of an inclusion proof and a consistency proof.

5.1.1. Inclusion proof

. The inclusion proof for the leaf $d[m]$ in $D[n]$, written $\text{PATH}(m, D[n])$ [23], gives the sibling hashes needed to recompute the root from that leaf. Generation of the inclusion proof is described in Appendix A.

To verify an inclusion proof, an auditor holding a leaf hash, its index m, the tree size n, the proof $\text{PATH}(m, D[n])$, and a trusted root recomputes the root using only these values. If the final tree hash matches the recorded root, the proof succeeds.

To allow an auditor to verify inclusion without access to the remainder of the log, the operator presents, for the entry at index m in a tree of size n:

```
{
  "index": m,
  "tree_size": n,
  "path": [h_3, h_01, h_4],
  "timestamp": "2026-07-30T09:58:20Z",
  "signature": "8d2998aa983...c30c49d"
}
```

The auditor already holds the trusted root and its tree size from the external anchor $\mathcal{R}$. The proof above is checked against the anchored root without revealing the other entries' content. Each element is a sibling subtree's hash, from which neither the sequences nor the entries beneath it can be recovered.

5.1.2. Consistency proof.

The consistency proof checks the current log $D[n_2]$ against the previous log $D[n_1]$ using $\text{PROOF}(n_1, D[n_2])$. By verifying the consistency proof, we learn if the tree of $D[n_2]$ is a valid extension of $D[n_1]$.

To let an auditor verify consistency without access to the rest of the log, the operator presents:

```
{
  "tree_size_1": 3,
  "tree_size_2": 5,
  "consistency_path": [h_2, h_3, h_01, h_4],
  "timestamp": "2026-07-30T09:58:20Z",
  "signature": "8d2998aa983...c30c49d"
}
```

For both cases, we require a receipt to be sent back to the benchtop by the same reasoning as in the section on anchoring.

5.2. Forensic Audit

These non-recurring audits are performed by $\mathcal{A}$ when tampering has been detected in a routine audit and they want to check for a SOC, or they have a specific sequence from an incident they want to assign to a benchtop. The sequences in $\mathcal{B}$ are decrypted with a private key that $\mathcal{A}$ holds, so they can see the raw sequences. In the case that $\mathcal{A}$ is checking for a specific sequence, they can run a standard searching algorithm like k-mer overlap or MinHash against the synthesis log.

During a forensic audit, physical inspection and tamper sensors can directly reveal tampering. Additionally, an audit of the TPM establishes hardware authenticity and reveals attempts to re-write past logs. During such an inspection, the auditor first provides a nonce to the TPM, and the module returns a `TPM2_NV_Certify` attestation of the TPM NV

index's current value, signed by the attestation key [17]. This certifies the hardware's authenticity and the legitimacy of the current index value. The auditor then recomputes every intermediate Merkle root, and uses these to replay the TPM index from V_0, resulting in a final index value. Comparing the computed final index value to the value reported by the TPM validates the exact Merkle root history. Any rollback, forking, or rewriting of any prior logs will either cause the checkpoints to fail to match the logs, or, if the adversary recomputed the checkpoints, will cause the TPM index value to fail to match the computed index value.

5.3. Provision

A provisioning protocol is required when B is initially provided to $\mathcal{O}$. The private signing key must be generated in the secure element, and its public key registered with $\mathcal{T}$. The public encryption key must be installed in the device, and the monotonic counter initialised. A connection should be established with the external anchor and the synthesis log initialised.

5.4. Transfer

When the benchtop is transferred between customers, the new customer shouldn't have access to previously synthesised sequences. The raw sequences are never held on the device, only their ciphertexts, whose private decrypting key is held by $\mathcal{A}$ and so sequences logged by previous customers are not available for the current operator to see. This defence holds regardless of whether the new operator's entries are appended to the same tree as the old operator or a different one. We recommend adding new sequences to the original tree due to ease of performing routine audit.

6. Implementation

We implement a prototype on a Lenovo ThinkStation with a TPM 2.0. It implements canonical serialisation, the Merkle tree over entries, per-append signed checkpoints, sequence encryption, and a running index enforced by the TPM. It is complementary to screening tools like those provided by SecureDNA.

Cryptography relies on the Python `cryptography` package (version 49.0.0). Everything else uses the standard library's `json` and `hashlib` libraries. The hashing function is SHA-256 and checkpoints are signed with Ed25519. We use a hybrid X25519 encryption scheme, storing the public encryption key on device.

The hardware path uses `tpm2-tools` (`tpm2_nvread`, `tpm2_nvextend`, `tpm2_nvreadpublic`, `tpm2_nvcertify`). Attestations are returned as a `TPMS_ATTEST` structure signed by a restricted ECC attestation key, which the auditor-side verifier parses field by field.

6.1. Performance

We profiled the implementation under a workload based on the synthesis timing of the Applied Biosystems 394, as documented in the manufacturer's user manual [26]. We selected this instrument because it was widely deployed and has publicly available timing documentation. However, it is an older instrument, and higher-throughput machines may place greater demands on the logging system. We assumed an average sequence length of 45 nucleotides and a conservatively high lifetime total of one million syntheses. Assuming approximately three hours per four-column run, this corresponds to about 86 years of continuous operation with all four columns occupied and no downtime for maintenance.

At the lifetime limit, appending a log entry took 6.5 s in software and was projected to take 13.3 s with TPM protection. Generating a fresh TPM audit proof took approximately 1.0 s and is not expected to depend materially on log length. Both operations are much faster than synthesizing a 45-nucleotide sequence. Verifying the latest authenticated state took 8.9 s at the lifetime limit. However, verifying every historical checkpoint scales quadratically and is projected to require approximately 2.3–2.6 days at the lifetime limit.

Profiling was performed on a consumer-grade Lenovo ThinkCentre. Computationally intensive auditing could be performed on more powerful auditor infrastructure, although eliminating the quadratic historical-checkpoint verification cost would require an algorithmic improvement.

6.2. Scope of prototype

The Merkle tree, checkpoint signatures, sequence encryption, and attestation parsing and verification are real implementations of the stated constructions, exercised end to end.

The device's Ed25519 signing key is an unencrypted file on disk, rather than on the secure element or TPM, so non-extractability isn't enforced. In a TPM on a benchtop device, the extend index is deleted with the signature from the auditor. The prototype uses a test index, which the machine's owner can delete.

A monotonic counter is not implemented in a secure element. The TPM index commits the tree size at each append so plays the counting role instead.

There is no mechanism to check if every physical synthesis event has reached the logging software. The design for such measures is beyond the scope of this paper.

6.3. Availability

The implementation is available at https://github.com/riothomas234/synthesis_log.

7. Theoretical security analysis

The append only attribute of the non volatile extend index of a TPM is enforced by a policy setting which would

TABLE 3. SECURITY PROPERTIES OF THE FULL DESIGN AND THEIR STATUS IN THE PROTOTYPE

Property	Status in prototype
Merkle tree, inclusion and consistency proofs	Implemented
Signed checkpoints	Implemented
Sequence encryption	Implemented
TPM extend index, replay and attestation	Implemented on a TPM 2.0, using a deletable test index
Non-extractable signing key (E1) and restricted signing (E2)	Requires a secure element; integrated in future hardware
Monotonic counter and append-only storage (E4)	Requires integration in future hardware
External anchor and receipts	Specified; not implemented
Binding of log entries to synthesis hardware (E5)	Out of scope

require physical attacks from an advanced actor to overcome [17]. The security of the private key in the TPM can be similarly guaranteed. If tamper evident seals are in place [13], an online connection can alert the authorities when the seals are broken. A state actor may not care about being caught, but these measures will ensure a moderate actor is detected and investigated. If tamper evident seals don't send online alerts on breaking, or append-only properties are not enforced due to the cost of implementing them, then the main line of defence maintaining the log's integrity is the merkle tree structure of the log and the storage of root hashes external to $\mathcal{B}$ for an auditor to check against. Now the remaining avenue of attack is the disruption of $\mathcal{B}$'s partial connection to $\mathcal{T}$. If $\mathcal{O}$ hides the disruption as a benign network failure, they may be able to edit synthesis log entries without detection, as the external anchor contains no root hash for an auditor to check against.

$\mathcal{O}$ receiving a receipt prevents the anchor from being modified and presented differently to an honest version of the synthesis log held on $\mathcal{B}$. It also defends against third party hackers breaching $\mathcal{T}$'s security and editing $\mathcal{R}$.

If a TPM is not implemented or the external connection is lost, a monotonic counter prevents undetectable deletion of previous entries and forking. An operator could try to reset the counter or fake counts on entries by attacks on the secure element like microprobing and side channel analysis [21].

Log entries include timestamps to avoid replay attacks and user IDs to prevent the copying of entries from another device that has legitimately printed a sequence of concern. If a threat actor breaches the connection of $\mathcal{B}$ to $\mathcal{T}$, they can inject forged entries. However, A2 is still intact, so such forged entries won't produce valid signatures. If the index of each entry is enforced by a monotonic counter, an attacker can't resend an old root hash to the external record with a valid signature because it will be evident that the entry's count is lower than present count. In normal use, the operator does not have total oracle access, i.e. E2 is enforced such that only appending of screened entries are signed.

The encryption scheme in $\mathcal{B}$ prevents third parties and future device owners from accessing sequences on the device. In addition, for a given plain text, each time it is encrypted with the public key, the ciphertext is different. However $\mathcal{T}$, with the private key, can always decrypt the ciphertext to find its sequence. This prevents an attacker from guessing sequences and checking if their ciphertexts match with those on the log, allowing them to learn sequences on device.

8. Related work

Secure logging approaches have been developed for different contexts before, inspiring many aspects of this design. However, the unique circumstance of DNA synthesis calls for novel additions.

Intel has a Software Guard Extensions (SGX) [27], [28] that limits access to certain data and code in a secure enclave to the code running in the enclave. As data moves to the RAM, it is encrypted and also given a cryptographic message authentication code (MAC) to detect modifications to it [29]. In order to prevent replay attacks, chunks of memory have version counters that increase as they are written. Each counter is a leaf in a tree whose root is kept in the CPU itself and used for verification, where attackers cannot reach it [29]. In a similar way, hashes of log entries containing encrypted DNA sequences form the leaves of a cryptographic Merkle tree in our design [22]. A hardware monotonic counter combined with timestamps on the external anchor act as version counters. Benchtop devices likely will not include custom CPUs, so these measures have to be enforced in addition to commercial systems. In SGX, keys are fused into the CPU [28] and remote attestation occurs [30]. Similarly, we propose signing keys be held in the secure element [20] and the TPM index be attested [17]. However, Intel's design aims to protect data belonging to a remote party from the machine operator, e.g. code being run on a remote server, whereas a secure logging design for benchtops prevents an operator from altering logs of synthesis events without an auditor detecting it. Hence, instead of computing MACs with a secret key so that only a CPU can check them, our design computes SHA-256 hashes [18] which anyone can recompute, allowing for third party auditing.

Solutions to tamper evident logging have been previously outlined by Crosby and Wallach [31], who considered the case of an untrusted logging service to which customers sent events to be logged. Previously, a log would have been kept as a hash chain with each entry carrying the hash of the previous entry, and customers could check if the chain was consistent and had not been tampered with. Walking through the whole chain is slow and a more efficient solution was suggested using a Merkle tree where customers hold root hashes and can ask for inclusion or consistency proofs from the logging service to check if the current log is a valid extension of the log they sent their event to [31].

TABLE 4. ATTACKS AVAILABLE TO $\mathcal{O}$ AND THE DEFENCE ADDRESSING EACH, UNDER TWO HARDWARE CONFIGURATIONS: AN EXTERNALLY ANCHORED MERKLE TREE WITH A SECURE ELEMENT, AND THE SAME DESIGN WITH A TPM NV EXTEND INDEX ADDED. *Capability* IS THE MINIMUM ATTACKER TIER (TABLE 2) REQUIRED TO DEFEAT THE STATED DEFENCE IN THE STRONGER CONFIGURATION.

Attack by $\mathcal{O}$	**Defence**	**Anchored + SE**	**+ TPM index**	**Capability to defeat**
Rewriting logged history				
Edit, delete, or reorder an entry	Every entry is a Merkle leaf; any change alters $\mathrm{MTH}(D[n])$, so the signed root no longer matches and the index replay diverges	Detected once anchored	Detected offline	Advanced
Truncate the tail	Anchored `tree_size` and `root_hash` record a longer log; the replayed index is short	Detected once anchored; exposed within λT	Detected offline	Advanced
Roll back to an earlier state	The extend-only index cannot be rewound undetectably; the monotonic counter cannot decrease	Detected via counter	Detected offline	Advanced
Fork the log or present a split view	Consistency proof fails against the anchored root; only one extend history can produce the certified value	Detected via consistency proof	Detected offline	Advanced
Attacking signing and identity				
Present another device's log as its own	`device_id` is bound into the anchored record and covered by the signature; the attestation key is device-unique	Detected	Detected	Advanced
Replay an earlier valid checkpoint from the same device	Anchored records carry $\mathcal{T}$-signed timestamps; $\mathcal{T}$ rejects a checkpoint whose `tree_size` and timestamp do not exceed the last anchored one.	Detected	Detected	Advanced
Extract a signing or attestation key	Keys generated inside the secure element and TPM and never exported; physical attack leaves evidence	Prevented, degrades	Prevented, degrades	Advanced
Delete or reset the NV index	Extend is the only write path; the deletion policy requires an auditor key	—	Prevented	Advanced
Attacking the anchor and its connection				
Sever the connection and tamper offline	Missed anchoring is an escalation trigger; the index witnesses the root history with no connection at all	Detected as anomaly; exposed within λT	Detected offline	Advanced
Alter the external anchor $\mathcal{R}$	$\mathcal{T}$-signed receipts retained by $\mathcal{O}$ contradict the altered record	Detected	Detected	Advanced
Attacking sequence confidentiality				
Recover logged sequences from $\mathcal{B}$	Hybrid public-key encryption; the decryption key never reaches the device	Prevented	Prevented	N/A
Residual				
Omit a synthesis event entirely	None; nothing on the device forces a synthesis run into the log or the index	**Not addressed**	**Not addressed**	**Moderate**
Inject a false entry in real time	None cryptographic; requires anti-tamper measures on the valve–controller path	**Not addressed**	**Not addressed**	**Moderate**

Certificate transparency logs [23], [24], which log the certificates granted to websites by certificate authorities so that certificates for a domain aren't misissued to those who don't control it, follow a similar concept.

Our design implements the same idea where the synthesis log is held by an untrusted benchtop operator, root hashes are held in a trusted location and an auditor requests consistency and inclusion proofs from the operator. Unlike the case specified by Crosby and Wallach [31] where events are appended by trusted clients, sequences are appended by a malicious operator, so we suggest an architecture with an external trusted auditor to hold root hashes and verify proofs instead. In addition to this, we encrypt DNA sequences on the device to maintain privacy and give a canonicalisation protocol that specifies entries to contain IUPAC nucleotide and ambiguity codes [32]. Crosby and Wallach assume that there is a complete online connection to the untrusted logger. For benchtop devices, we are careful to specify the minimum information necessary to send from a device to an auditor, so that partial internet connections can be maintained in an attempt to avoid leaking private DNA sequences. In addition to this, we specify hardware protections against tampering for devices that can maintain no internet connection at all.

9. Discussion

9.1. Limitations

There are several possible attacks which the present work does not address, such as adding false entries in real time by manipulating the connection between valves and software [13], or omitting synthesis events outright. So,

while a forensic audit that finds a sequence of concern in a benchtop device's synthesis log counts as strong evidence for malicious activity, an audit that finds none is weak evidence of innocence. As Table 4 shows, omitting a synthesis event only requires moderate ability rather than the advanced ability required to rewrite logged history. A malicious actor looking to synthesize a sequence of concern will take the cheaper route. However, the design proposed in this paper raises the difficulty of tampering a log from simply modifying past synthesis entries to interfering with the device during a synthesis event. The latter attacks are more likely to leave physical traces that tamper seals and inspection can reveal [13].

Further measures can be taken to reduce the security gap posed by omission. A separate tamper-resistant chip that counts the number of valve actuations or nucleotide additions lets an auditor compare the length of the synthesis log against the amount of synthesis performed. Furthermore, reagent consumption can be measured using the cartridge [13] or reagent orders recorded by the manufacturer. A wide variety of sensors might also detect when synthesis occurs but logging doesn't.

9.2. Deployment

Our design requires very little information to be sent to the internet, i.e. one signed record (device ID, tree size, root, timestamp, signature), that is a few hundred bytes. This makes low bandwidth internet connections viable. Of course, many commercial benchtops may be connected to computers with full internet access [9], but for customers who desire higher privacy, lower bandwidth options can improve privacy while maintaining biosecurity. For example, an embedded SIM over a cellular network is independent of the operator. However, it requires that the benchtop provider add an additional module to their device and have a data contract. This limited connection can be broken when shielded or jammed but doing so would be more conspicuous than breaking a connection over a router. The connection does not have to be ongoing; anchoring once every day shrinks the window an operator can synthesize a sequence of concern and remove it from the log. Fully airgapped labs may completely rely on the TPM index and physical inspection for secure logging which is expensive. Legislation aimed at preventing split-order attacks [25] may also require that benchtops be internet connected, partly obviating the need for a TPM.

However, it can be costly to integrate a TPM into the firmware of a benchtop and perform factory provisioning. Other protections to detect hardware tampering like tamper meshes and sensors [13] add to the cost of the device, as well as measures to secure the interface between synthesiser valves and firmware. Maintaining a trusted location to hold external anchored checkpoints as well as upkeeping an auditing authority to verify proofs leads to an ongoing cost. Physical inspections for offline devices also cost time and money. Furthermore, financially valuable sequences can only be held in this system only if the auditor's private key is truly secure and backed up in a secure location.

9.3. Governance

The auditing authorities can perform routine audits at regular intervals, requesting audit proofs from the benchtop that avoid disclosing intellectual property. They have the record of root hashes to check against from our trusted location. Existing biosecurity organizations like IBBIS [33] or SecureDNA [34] could potentially serve as auditors themselves, or otherwise build the governance infrastructure required to perform auditing.

10. Conclusion

At the start of this paper, we aspired to achieve the goals set out in 'Protein design meets biosecurity' [5], for benchtop devices. We suggested enforcing append-only properties in benchtops on the hardware level, but in the case that these features are not implemented due to cost, proposed a cloud-based architecture involving an external auditing authority and a trusted location to hold cryptographic data. We recommended routine audits to happen at a regular cadence, with no disclosure of proprietary intellectual property, maintaining trade secrets. A 'selective revelation' policy is captured in our forensic audit protocol, where auditors can search for raw sequences on benchtops in response to a biological threat, as part of a traceback regime. Such an audit might happen on the basis of preregistered criteria which we expect to be created by standard setting bodies like the SBRC [35] and EBRC [36]. The awareness of an audit trail will deter bad actors from the synthesis of sequences of concern [5]. An international group like the IGSC [37] should lead the building of infrastructure to perform audits and hold secure data. This requires collaborating with governmental and non-governmental bodies so that collection of data from benchtops provided by different companies can take place.

11. Acknowledgements

We would like to thank Tessa Alexanian (IBBIS), Rami Puzis (Ben Gurion University) and Lee Wall for their continuous feedback on this manuscript. This work was carried out with financial support from the ERA AIxBio fellowship.

Appendix A. Inclusion Proof Generation

We write :: for list concatenation when appending a node hash to an audit path.

For a single-element log, the proof is empty [23]:

$$\mathrm{PATH}(0, \{d[0]\}) = \{\} \tag{2}$$

For $n > 1$, with k the largest power of two strictly less than n (as in (1)), it is defined recursively:

$$\mathrm{PATH}(m, D[n]) = \begin{cases} \mathrm{PATH}(m, D[0{:}k]) :: \mathrm{MTH}(D[k{:}n]) & m < k \\ \mathrm{PATH}(m-k, D[k{:}n]) :: \mathrm{MTH}(D[0{:}k]) & m \geq k \end{cases} \tag{3}$$

Appendix B. Inclusion Proof Verification

Let r be a running hash, initialised to the leaf hash, and let $i = m$ and $\ell = n - 1$ where m is the index of the leaf hash whose inclusion we are checking and n is the tree size. For each sibling p in the proof, taken in order from leaf to root, r is updated according to whether the current node is a left or right child [23]:

$$r \leftarrow \begin{cases} \mathrm{HASH}(\texttt{0x01} \parallel p \parallel r) & \text{if } i \text{ is odd, or } i = \ell \\ \mathrm{HASH}(\texttt{0x01} \parallel r \parallel p) & \text{otherwise} \end{cases} \tag{4}$$

after which both indices are halved, $i \leftarrow \lfloor i/2 \rfloor$ and $\ell \leftarrow \lfloor \ell/2 \rfloor$, ascending one level in the tree. When i is odd, the node represented by r is a right child, so its sibling hash p is placed on the left when hashing both nodes combined. If i is even, r is a left child and p is placed on the right, unless i corresponds to the rightmost node in the layer, in which case r is a right child. Because the order of text within the hash function matters to its output, the left/right decision is essential to correctness.

Appendix C. Consistency Proof Generation

The consistency proof is a list of hashes you combine the tree hash of the previous tree with to form your current tree. RFC 6962 [23] gives the procedure to generate the proof below, initialising with b as True:

$$\mathrm{PROOF}(m, D[n]) = \mathrm{SUBPROOF}(m, D[n], \textsc{true}). \tag{5}$$

$$\mathrm{SUBPROOF}(m, D[m], \textsc{true}) = \{\} \tag{6}$$

$$\mathrm{SUBPROOF}(m, D[m], \textsc{false}) = \{\, \mathrm{MTH}(D[m]) \,\}. \tag{7}$$

For $m < n$, let k be the largest power of two strictly less than n. The subproof recurses into whichever subtree contains the size-m boundary and calculates the tree hash of the other. Writing S for SUBPROOF, if $m \leq k$:

$$S(m, D[n], b) = S(m, D[0{:}k], b) \parallel \mathrm{MTH}(D[k{:}n]), \tag{8}$$

and if $m > k$:

$$S(m, D[n], b) = S(m{-}k, D[k{:}n], \textsc{false}) \parallel \mathrm{MTH}(D[0{:}k]). \tag{9}$$

Appendix D. Consistency Proof Verification

Let $\pi = \mathrm{PROOF}(m, D[n])$ be the proof produced by generation and handed to the auditor [23]. Let k be the largest power of two strictly less than the current size n, and let $\textsc{next}(\pi)$ take the next unused element of π. At the boundary $m = n$:

$$\mathrm{V}(m, m, \textsc{true}) = \text{old_root}, \quad \Pi \mathrel{+}= \text{old_root} \tag{10}$$

$$\mathrm{V}(m, m, \textsc{false}) = p, \quad \Pi \mathrel{+}= p, \quad \text{where } p = \textsc{next}(\pi) \tag{11}$$

For $m < n$, if $m \leq k$:

$$\mathrm{V}(m, n, b) = \mathrm{HASH}\big(\texttt{0x01} \parallel \mathrm{V}(m, k, b) \parallel \textsc{next}(\pi)\big), \tag{12}$$

and if $m > k$:

$$\mathrm{V}(m, n, b) = \mathrm{HASH}\big(\texttt{0x01} \parallel p \parallel \mathrm{V}(m{-}k,\, n{-}k,\, \textsc{false})\big), \quad \Pi \mathrel{+}= p \tag{13}$$

Two checks must then both hold. The reconstruction must equal the new root, with π fully consumed:

$$\mathrm{V}(m, n, \textsc{true}) = \text{new_root}. \tag{14}$$

Independently, the peaks in Π — taken largest-first — must recombine to the old root, right-associatively:

$$\text{old_root} = \mathrm{HASH}\big(\texttt{0x01} \parallel P_1 \parallel \mathrm{HASH}(\texttt{0x01} \parallel P_2 \parallel \cdots \mathrm{HASH}(\texttt{0x01} \parallel P_{t-1} \parallel P_t) \cdots)\big), \tag{15}$$

for Π ordered $P_1, \ldots, P_t$ from largest to smallest subtree.

The second check is necessary, not implied by the first. When m is a power of two, old_root enters the reconstruction of new_root directly and the first check pins it down. When m is not a power of two, old_root is combined with sibling material and never revisited, so the reconstruction can succeed for an incorrect old_root; the peak recombination (15) is what closes this. Note that this recombination is different to if we treated each peak as a leaf hash of a new tree. That method would diverge from (15) whenever $t \geq 3$: for $m = 7$, peaks of size $4, 2, 1$ must combine as $\mathrm{HASH}(P_4, \mathrm{HASH}(P_2, P_1))$, not $\mathrm{HASH}(\mathrm{HASH}(P_4, P_2), P_1)$.

References

[1] D. Listov, C. A. Goverde, B. E. Correia, and S. J. Fleishman, "Opportunities and challenges in design and optimization of protein function," *Nature Reviews Molecular Cell Biology*, vol. 25, no. 8, pp. 639–653, 2024.

[2] M. Langenkamp, "Securing benchtop DNA synthesizers," Institute for Progress, Tech. Rep., Dec. 2024, https://ifp.org/securing-benchtop-dna-synthesizers/, accessed 2026-08-05.

[3] G. Alvarado-Urbina, G. M. Sathe, W.-C. Liu, M. F. Gillen, P. D. Duck, R. Bender, and K. K. Ogilvie, "Automated synthesis of gene fragments," *Science*, vol. 214, no. 4518, pp. 270–274, 1981.

[4] Y. Ma, Z. Zhang, B. Jia, and Y. Yuan, "Automated high-throughput dna synthesis and assembly," *Heliyon*, vol. 10, no. 6, 2024.

[5] D. Baker and G. Church, "Protein design meets biosecurity," pp. 349–349, 2024.

[6] "AI executives join call for stricter regulation of synthetic biology." [Online]. Available: https://www.science.org/content/article/ai-executives-join-call-stricter-regulation-synthetic-biology

[7] "Screening Framework Guidance for Providers and Users of Synthetic Nucleic Acids (October 2023)."

[8] "UK screening guidance on synthetic nucleic acids for users and providers." [Online]. Available: https://www.gov.uk/government/publications/uk-screening-guidance-on-synthetic-nucleic-acids/uk-screening-guidance-on-synthetic-nucleic-acids-for-users-and-providers

[9] Nuclear Threat Initiative, "Benchtop DNA synthesizers: An assessment of risks and benefits for research settings," Nuclear Threat Initiative, Tech. Rep., May 2023, https://www.nti.org/wp-content/uploads/2023/05/NTIBIO_Benchtop-DNA-Report_FINAL.pdf, accessed 2026-08-05.

[10] R. Gerhards, "The Syslog Protocol," Internet Engineering Task Force, Request for Comments RFC 5424, Mar. 2009, num Pages: 38. [Online]. Available: https://datatracker.ietf.org/doc/rfc5424

[11] A. T. Sherman, J. J. R. Romano, E. Zieglar, E. Golaszewski, J. D. Fuchs, and W. E. Byrd, "Analysis of the Security Design, Engineering, and Implementation of the SecureDNA System," 2025.

[12] C. Baum, J. Berlips, W. Chen, H. Cozzarini, H. Cui, I. Damgård, J. Dong, K. M. Esvelt, L. Foner, M. Gao, D. Gretton, M. Kysel, J. Li, X. Li, O. Paneth, R. L. Rivest, F. Sage-Ling, A. Shamir, Y. Shen, M. Sun, V. Vaikuntanathan, L. V. Hauwe, T. Vogel, B. Weinstein-Raun, Y. Wang, D. Wichs, S. Wooster, A. C. Yao, Y. Yu, H. Zhang, and K. Zhang, "A system capable of verifiably and privately screening global DNA synthesis," Jun. 2025, arXiv:2403.14023 [cs.CR]. [Online]. Available: http://arxiv.org/abs/2403.14023

[13] "Securing Benchtop DNA Synthesizers | IFP," Dec. 2024, section: Biotechnology. [Online]. Available: https://ifp.org/securing-benchtop-dna-synthesizers/

[14] J. Saltzer and M. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, Sep. 1975. [Online]. Available: https://ieeexplore.ieee.org/document/1451869/

[15] G. H. Kim and E. H. Spafford, "The design and implementation of tripwire: a file system integrity checker," in *Proceedings of the 2nd ACM Conference on Computer and communications security*, ser. CCS '94. New York, NY, USA: Association for Computing Machinery, Nov. 1994, pp. 18–29. [Online]. Available: https://dl.acm.org/doi/10.1145/191177.191183

[16] W. Arbaugh, D. Farber, and J. Smith, "A secure and reliable bootstrap architecture," in *Proceedings. 1997 IEEE Symposium on Security and Privacy (Cat. No.97CB36097)*, May 1997, pp. 65–71, iSSN: 1081-6011. [Online]. Available: https://ieeexplore.ieee.org/document/601317/

[17] "TPM 2.0 Library." [Online]. Available: https://trustedcomputinggroup.org/resource/tpm-library-specification/

[18] N. I. o. S. and Technology, "Secure Hash Standard (SHS)," U.S. Department of Commerce, Tech. Rep. Federal Information Processing Standard (FIPS) 180-4, Aug. 2015.

[19] S. Josefsson and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)," Internet Engineering Task Force, Request for Comments RFC 8032, Jan. 2017.

[20] "ATECC608A," https://www.microchip.com/en-us/product/atecc608a.

[21] T. Roche, "EUCLEAK," 2024.

[22] R. C. Merkle, "A Certified Digital Signature," in *Advances in Cryptology — CRYPTO' 89 Proceedings*, G. Brassard, Ed. New York, NY: Springer, 1990, pp. 218–238.

[23] B. Laurie, A. Langley, and E. Kasper, "Certificate Transparency," Internet Engineering Task Force, Request for Comments RFC 6962, Jun. 2013, num Pages: 27. [Online]. Available: https://datatracker.ietf.org/doc/rfc6962

[24] B. Laurie, E. Messeri, and R. Stradling, "RFC 9162: Certificate transparency version 2.0," IETF Request for Comments, 2021, https://datatracker.ietf.org/doc/html/rfc9162.

[25] R. Edison, S. Toner, and K. M. Esvelt, "Assembling unregulated DNA segments bypasses synthesis screening: Regulate fragments as select agents," *Nature Communications*, vol. 17, no. 1, p. 3189, Jan. 2026.

[26] Applied Biosystems, Inc., *Models 392 and 394 DNA/RNA Synthesizers: User's Manual*, May 1991, revision C, Part No. 901237. https://documents.thermofisher.com/TFS-Assets/LSG/manuals/cms_095578.pdf.

[27] F. McKeen, I. Alexandrovich, A. Berenzon, C. V. Rozas, H. Shafi, V. Shanbhogue, and U. R. Savagaonkar, "Innovative instructions and software model for isolated execution," in *Proceedings of the 2nd International Workshop on Hardware and Architectural Support for Security and Privacy (HASP)*, 2013.

[28] V. Costan and S. Devadas, "Intel SGX explained," Cryptology ePrint Archive, Paper 2016/086, 2016. [Online]. Available: https://eprint.iacr.org/2016/086

[29] S. Gueron, "A memory encryption engine suitable for general purpose processors," Cryptology ePrint Archive, Paper 2016/204, 2016. [Online]. Available: https://eprint.iacr.org/2016/204

[30] I. Anati, S. Gueron, S. P. Johnson, and V. R. Scarlata, "Innovative technology for CPU based attestation and sealing," in *Proceedings of the 2nd International Workshop on Hardware and Architectural Support for Security and Privacy (HASP)*, 2013.

[31] S. A. Crosby and D. S. Wallach, "Efficient Data Structures for Tamper-Evident Logging."

[32] A. Cornish-Bowden, "Nomenclature for incompletely specified bases in nucleic acid sequences: Recommendations 1984," *Nucleic Acids Research*, vol. 13, no. 9, pp. 3021–3030, 1985.

[33] "IBBIS - Safeguarding modern bioscience and biotechnology," https://ibbis.bio/.

[34] "Safeguarding DNA synthesis – SecureDNA," https://securedna.org/.

[35] "SBRC | Sequence Biosecurity Risk Consortium," https://screening-testing.github.io/sbrc-website/.

[36] "EBRC | An inclusive community of researchers committed to advancing engineering biology to address national and global needs," https://ebrc.org/.

[37] "Home | International Gene Synthesis Consortium," Oct. 2017. [Online]. Available: https://genesynthesisconsortium.org/